ELI — Dag Orchestrator

ELI v2.5.10

October 2026

Contents

DAG orchestrator — project-wide execution engine (2026-06-07) **1**
The orchestrator (`eli/core/dag.py`) . . . 1
Wired through the pipeline . . . 2
Tests . . . 2
Test/project run → review (GUI + chat) . . . 2
Generative actions on the coding agent / planner DAG . . . 2
SELF_IMPROVE routed through the coding agent . . . 2
Multi-command + timed commands (2026-06-08) . . . 3

DAG orchestrator — project-wide execution engine (2026-06-07)

`eli/core/dag.py` was a pure **scheduling** primitive (Kahn topological order, parallel layers, cycle detection, ancestors/descendants, critical-path). It was elevated into a full **execution orchestrator** and wired through the pipeline — additively, so every existing class/function and caller is unchanged.

The orchestrator (`eli/core/dag.py`)

`Orchestrator` / `run_graph(tasks)` execute a set of `Task`s on their dependency DAG:

- **Parallel intra-layer execution** — independent nodes in a topological layer run concurrently (threads); `max_workers=1` ⇒ deterministic sequential.
- **Dependency result-passing** — each node fn gets a `NodeContext` with `results` (completed upstream outputs) + a shared mutable `shared` bag.
- **Conditional nodes** — `when(ctx)->bool` skips a branch dynamically.
- **Retries + backoff** — per-node `retries` with exponential `retry_backoff`.
- **Fallback** — a `fallback` fn runs if all attempts fail.
- **Per-node timeout** — total node budget (best-effort; enforced via a worker future).
- **Memoisation** — pluggable `cache` keyed by `cache_key` (status `cached`).
- **Priority scheduling** — higher-priority nodes first within a ready layer.
- **Fail-fast + global time budget** — stop scheduling after a failure / budget.
- **Auto-skip** — a node whose upstream didn't complete is skipped with a reason.
- **`critical` semantics** — a non-critical node's failure doesn't fail the run.
- **Deterministic `RunReport`** — per-node status/attempts/timing + order + layers + critical path + failed/skipped lists + `to_dict()` for observability.

Pure stdlib; the orchestrator does no LLM/IO — the node callables do.

Wired through the pipeline

- **Agents (cognition):** the agent bus runs the 15 agents on the orchestrator (`_run_agents_orchestrated`, default `ELI_AGENT_ORCHESTRATOR=1`, falls back to the layered then flat paths). Dependency-ordered, intra-layer parallel, per-agent timeouts (mode + model-tier scaled), upstream results passed downstream, and a `RunReport` captured per dispatch. Behaviour-preserving: agent fns never raise out (errors → `ok=False`), so edges only order + pass upstream, never gate.
- **Router → executor (observability):** `ORCHESTRATION_STATUS` action ("orchestration status" / "show your agent dag") returns the live engine, execution layers, dependencies, critical path, and last-run telemetry — so ELI can explain its own orchestration honestly.
- **Evidence gathering:** `evidence_planner.gather()` now runs its channels (code / web / memory / runtime) **in parallel on the orchestrator** — each channel isolated (non-critical: one failing yields no evidence, never blocks others), results merged in deterministic `KNOWN_CHANNELS` order, with a sequential fallback. `memory`/`runtime` (no model calls) genuinely overlap; model-using channels serialise safely on the global `_LLM_CALL_LOCK`.
- **report_pipeline — deliberately NOT parallelised.** All model calls serialise on `_LLM_CALL_LOCK`, so parallel section drafting gives zero speedup (model-bound + globally locked) while adding risk. It stays sequential with its own retries.
- **GUI:** a read-only **Orchestration** sub-tab in Labs (and the `ORCHESTRATION_STATUS` chat action) surface the live layers/deps/critical-path + the last dispatch's per-node RunReport.
- Also on the DAG: the coding planner (`coding/plan_graph.py`) and the LoRA pipeline.

Tests

`tests/test_dag_orchestrator.py` (14) cover every feature; existing dag/coding/agent tests + the full suite stay green. Flags: `ELI_AGENT_ORCHESTRATOR`, `ELI_AGENT_DAG`.

Test/project run → review (GUI + chat)

`eli/runtime/test_review.py::run_and_review()` runs the suite (or a subset), **backs up the prior report**, **writes a timestamped errors file** capturing failing tests (possible errors) under `artifacts/test_reports/`, and returns a **results-driven options menu** (each option carries a chat `command` routing to examine/propose/ generate/eval). Surfaced two ways: - **GUI:** a " Test & Review" Labs sub-tab — run the full suite in the background, see totals + failures + backup/error paths, get **ELI's natural-language summary**, and click result-driven option buttons that hand off to ELI in chat. - **Chat:** the `TEST_REVIEW` action ("test review" / "run the tests and tell me what to fix") returns the grounded report (LLM-summarised) + the options menu. - **Option buttons stream into the main chat** (`MainWindow.send_to_chat`) — clicking an option continues the conversation in the Chat tab, not the panel.

Generative actions on the coding agent / planner DAG

- **GENERATE_PROJECT** → `CodeAgent.solve(use_dag=True)`: decompose into a subtask DAG (`plan_graph` over `eli/core/dag`), solve nodes in order, VERIFY the composed module; saves to `artifacts/projects/`. Single-pass `chat` fallback (`ELI_PROJECT_DAG=0`).
- **GENERATE_SCRIPT / CREATE_SCRIPT / WRITE_SCRIPT** → already routed through the verified coding agent (plan → DAG/tree-search → execute → repair → bug-memory).

SELF_IMPROVE routed through the coding agent

`SelfImprovementEngine.propose_via_agent()` (mode `propose`, or auto-detected from "propose/verified fix/fix the failing") turns each recent failure into a coding task and runs the **CodeAgent** (decompose→solve→**verify** via tree-search + execution gate) over them **in parallel on `run_graph`**, returning **verified, propose-only** fixes (gated; nothing applied until "apply self-improvement patch"). This is the delegation pattern realised: SELF_IMPROVE composes the existing coding agent + orchestrator rather than reinventing repair. The Test & Review "Propose verified fixes" option routes here.

Artifact-dir resolution is now consistent — `run_test_report.py` and the conftest report hook both honour `ELI_ARTIFACTS_DIR` (no real-folder pollution in tests).

Multi-command + timed commands (2026-06-08)

- **Multi-command chaining** (`eli/runtime/command_splitter.py` + router `multi_command_prepass` → `MULTI_COMMAND`): one utterance chaining imperative commands is split (the "plan"), then the executor routes & runs each segment in order and combines results — the planning step that lets the agents/executor handle several actions from one sentence. Sequential today; `run_graph` is ready for parallel fan-out of independent ("and"-joined) commands.
- **Timed commands** → background workers: any imperative + a future time defers to the durable scheduler (see `runtime_planning_world.md`).